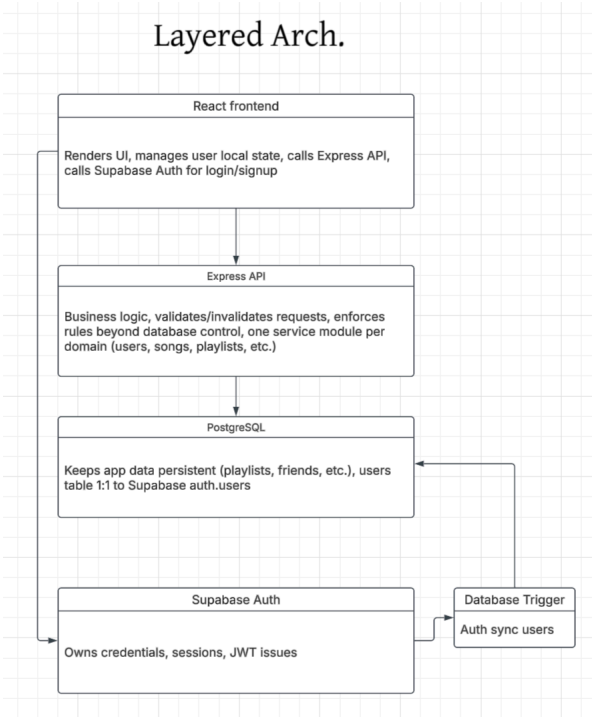
**C.L.R - Andy Cai, Jonathan Resendiz**
**TE4: Identify Design and Architectural Patterns**

1) **Proposed Architecture**

The primary architecture pattern our application will use is layered architecture. We chose layered architecture because we prioritized maintainability and security over pure performance and scalability. Because it's at a class project size, AuxShare's current scale doesn't demand huge scale and real time features won't be a limitation.

Layered architecture also can model our project flow well, it models how the frontend leads to an Express API to the database, which will also help us decide where new features and pieces of code belong to easier, and also help us identify where an error or bug is coming from.




**AuxShare Proposed Architecture**

React frontend - Renders UI, manages user local state, calls Express API, calls Supabase Auth for login/signup

↓

Express API - Business logic, validates/invalidates requests, enforces rules beyond database control, one service module per domain (users, songs, playlists, etc.)

↓

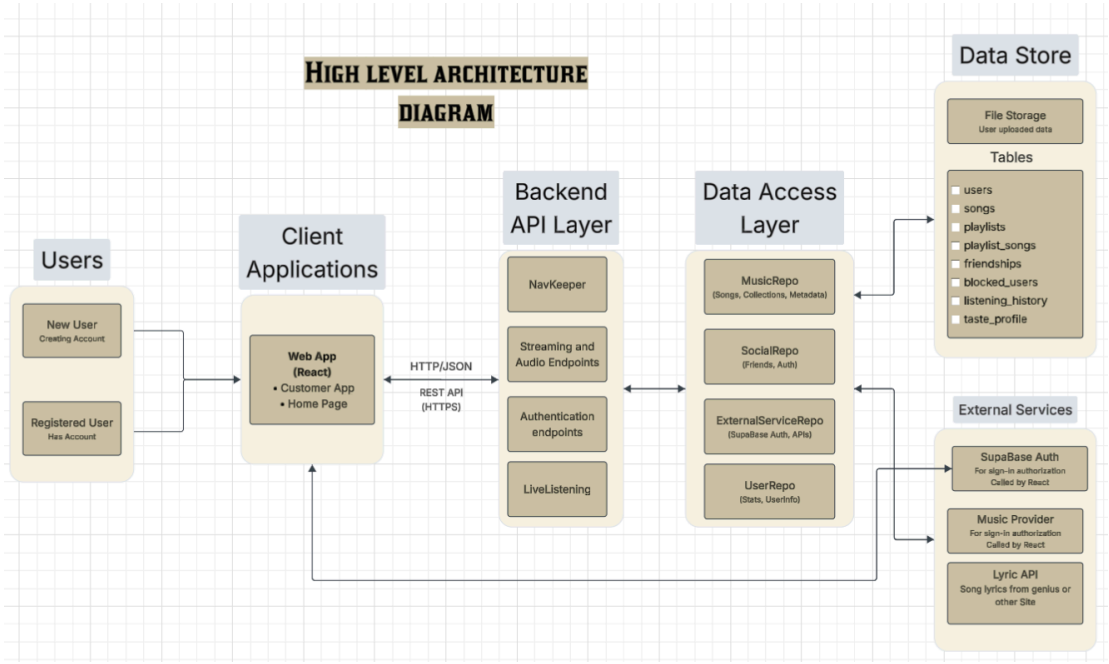
PostgreSQL - Keeps app data persistent (playlists, friends, etc.), users table 1:1 to Supabase auth.users

↑

Supabase Auth - Owns credentials, sessions, JWT issues [Auth sync users]

**The external system is Supabase Auth being the authentication service for user login/sign-up called from React frontend**

## 2) High Level Design Diagram

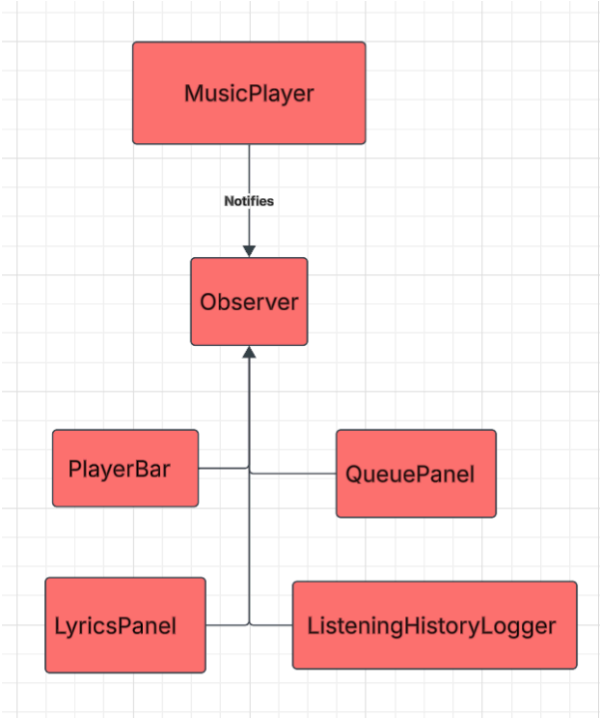


3) **Design Patterns**

**The design pattern we're using is observer and the problem it solves is the UI updating everytime there is an update with the players contents. Every time the music player updates such as changing the current song, play/pause state, the playing time(updating consistently), a few parts of the UI the user sees on their page has to update. With the observer pattern implemented, we can just have the music player call notify and the components subscribe to the music player which calls notify(). Otherwise the music player would have to call and make changes to every component that updates individually.**

**[This has not been implemented yet but planned to be implemented in the updating of the musicplayer UI]**

**Components: MusicPlayer (subject) notifies QueuePanel, LyricsPanel, PlayerBar, and ListeningHistoryLogger (observers).**



## 4) Design Decisions and Tradeoffs

**Chosen Designs**

**1) Using Supabase Auth rather than building authentication from scratch**

**We originally wanted to keep a password hash with the user component and check the hash whenever they logged in, but we transitioned to Supabase Auth since it's secure and we can spend more time on other components. The single tradeoff to this choice is just that we have less controls over the auth internals.**

**2) Implement Observer pattern into our music player components**

**We decided to use the observer pattern because the music player is constantly updating and several components depend on its state. The alternative we considered was having MusicPlayer call each component directly. With Observer, the music player only calls notify(), and each subscribed component updates itself, so the player doesn't need to know which components exist. The tradeoff is that the data flow is less explicit, which makes debugging harder, and components must unsubscribe when they unmount to avoid memory leaks.**